A Reproducible Benchmarking Framework for Evaluating TLS and mTLS Performance Overhead in Zero Trust-Aligned Apache Kafka Deployments

Nick Efe Oni

Supervisor: Mahshid Delavar

A report submitted in fulfilment of the requirements for the degree of MEng in Software Engineering

in the

School of Computer Science

April 20, 2026

Declaration

All sentences or passages quoted in this report from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). Any illustrations that are not the work of the author of this report have been used with the explicit permission of the originator and are specifically acknowledged. I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this project and the degree examination as a whole.

Name: Nick Efe Oni

Signature: Nick Efe Oni

Date: April 20, 2026

Abstract

an abstract is provided to give a brief overview and summary of the core topics, arguments, and findings . Because it must summarize your entire project—including your final benchmarking results from Chapter 5 and your final conclusions from Chapter 7—you should write the abstract at the very end of the process, once the rest of your dissertation is completely finished.

Contents

1 Introduction **1**
1.1 Background and Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Aim and Objectives . . . 2
1.4 Research Question . . . 3
1.5 Artefact Overview . . . 3
1.6 Dissertation Structure . . . 4

2 Literature Review **5**
2.1 Introduction to Literature Review . . . 5
2.2 Apache Kafka & Distributed Streaming Systems . . . 6
2.3 Performance in Kafka Systems . . . 7
2.4 Security Mechanisms in Kafka . . . 9
2.5 Zero Trust Architecture . . . 10
2.6 Zero Trust in Distributed Streaming Systems . . . 11
2.7 TLS/mTLS Performance Overhead . . . 13
2.8 Benchmarking Methodologies . . . 14
2.9 Synthesis & Research Gap . . . 15

3 Design, Implementation, and Testing **18**
3.1 Chapter Overview . . . 18
3.2 Artefact Requirements . . . 19
3.3 System Architecture . . . 20
3.4 Infrastructure Provisioning . . . 21
3.5 Kafka Cluster Deployment . . . 23
3.6 Security Mode Design . . . 24
3.7 Benchmark Workload Design . . . 25
3.8 Factorial Experiment Design . . . 26
3.9 Execution and Orchestration . . . 28
3.10 Telemetry and Metric Collection . . . 29
3.11 Result Processing and Export . . . 30
3.12 Fault Tolerance and Idempotency . . . 31

3.13 Testing and Validation 32
3.14 Chapter Summary 34

4 Results and Discussion **36**

5 Evaluation, Conclusions, and Further Work **37**

Appendices **40**

A Ethics and GenAI Declaration **41**
A.1 Ethics Statement 41
A.2 Generative AI Declaration 41
A.2.1 Acknowledge 41
A.2.2 Describe 42
A.2.3 Evidence 42
A.2.4 Declaration 43

B Artefact Configuration **44**

C Result Schema **45**

D Experiment Plans **46**

E Additional Result Tables **47**

F Repository and Reproducibility Notes **48**

List of Figures

2.1 Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026. 7

2.2 Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use `sendfile()` to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user space for encryption before transmission. 9

2.3 Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication. 10

2.4 Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary. 13

3.1 High-level system architecture of the Kafka benchmarking artefact. 21

List of Tables

3.1 Core artefact requirements 20
3.2 Kafka transport-security modes evaluated by the artefact 25
3.3 Testing and validation activities used for the benchmarking artefact 34

A.1 Representative examples of GenAI use during the project 43

Chapter 1

Introduction

1.1 Background and Motivation

Modern distributed systems increasingly rely on event-streaming platforms to connect cloud services, data pipelines, and enterprise applications [1, 2]. Apache Kafka is a widely used open-source event-streaming platform that supports high throughput, low latency, fault tolerance, and horizontal scalability [3–5]. These properties make Kafka a critical component in data-driven systems where large volumes of events must move reliably between producers, brokers, and consumers.

Kafka achieves much of its performance through partition-level parallelism, batching, append-only logs, page-cache utilisation, and zero-copy data transfer [3, 6, 7]. However, these mechanisms also make Kafka sensitive to additional processing in the data path. Security mechanisms are therefore not only deployment details, as they can affect the communication and computation paths that Kafka relies on for high-throughput streaming.

At the same time, distributed systems face security risks that traditional perimeter-based models do not fully address. In cloud and multi-service environments, internal network access can no longer be treated as inherently trustworthy, as insider threats and lateral movement attacks can originate from within the network boundary [8–10]. This has increased interest in Zero Trust Architecture, whose central principle is commonly summarised as "never trust, always verify" [8–11]. Zero Trust deployments usually emphasise strong identity, mutual authentication, least-privilege access, encrypted communication, monitoring, and policy-based control [9, 11, 12].

Zero Trust Architecture is not a single Kafka feature. It is a broader system-level security paradigm. However, Kafka provides native mechanisms that align with several Zero Trust principles, including transport encryption, certificate-based authentication, broker-to-broker security, SASL authentication, and ACL-based authorisation [7, 13–15]. Among these, TLS and mutual TLS (mTLS) are particularly relevant because they

directly affect Kafka's data path and allow plaintext, server-authenticated, and mutually authenticated transport configurations to be compared.

1.2 Problem Statement

Existing research provides strong foundations for both Zero Trust Architecture and Apache Kafka performance. Zero Trust has been discussed in distributed and streaming systems [8,12,16], while Kafka has been widely analysed as a high-throughput event-streaming platform [4, 5, 17]. However, there is still limited systematic empirical evaluation of the performance impact of Zero Trust-aligned transport security in Kafka-based environments.

This gap matters because TLS and mTLS introduce cryptographic processing into communication paths that Kafka is designed to keep efficient. Kafka's official design documentation states that SSL/TLS prevents the use of the `sendfile` zero-copy optimisation because SSL libraries operate in user space [6]. Wider TLS literature also shows that secure transport can introduce computational and latency overhead, although the scale depends on workload, connection behaviour, and system configuration [18–20]. Therefore, the cost of TLS and mTLS cannot be assumed to be constant across Kafka deployments.

The unresolved problem is the lack of a unified, reproducible, and Kafka-specific framework for measuring this overhead under controlled configurations. In particular, there is limited evidence showing how plaintext, TLS, and mTLS compare when workload and deployment parameters are matched, and how this overhead changes across Kafka variables such as broker count, replication factor, acknowledgements, batching, compression, producer concurrency, and message size.

1.3 Research Aim and Objectives

The aim of this dissertation is to design, implement, and evaluate a reproducible benchmarking framework for quantifying the performance overhead introduced by TLS and mTLS in high-throughput Apache Kafka deployments.

The objectives are:

1. To analyse Kafka's performance mechanisms and identify how transport security may affect them.
2. To design and implement a cloud-based Kafka benchmarking artefact supporting plaintext, TLS, and mTLS.
3. To provision reproducible Kafka deployments using infrastructure-as-code and automated orchestration.

4. To execute matched benchmark workloads across security modes using explicit configuration files.
5. To measure throughput and latency for each benchmark run.
6. To collect host-level telemetry from Kafka brokers and benchmark clients, including CPU, memory, disk, and network activity.
7. To evaluate how TLS and mTLS overhead varies across Kafka workload and deployment parameters.
8. To include targeted consumer-side validation to assess whether secure transport also affects Kafka's read path.

1.4 Research Question

This dissertation addresses the following research question:

> "What is the performance overhead introduced by Zero Trust-aligned security mechanisms, particularly mTLS, in high-throughput Kafka-based distributed systems?"

1.5 Artefact Overview

To answer this research question, this dissertation designs, implements, and evaluates a configurable Kafka benchmarking artefact. The artefact provisions Apache Kafka clusters on AWS EC2 using Terraform, deploys Kafka in KRaft mode, and supports three transport-security configurations: plaintext, TLS, and mTLS. Benchmark execution is driven by structured configuration files so that equivalent workloads can be repeated across security modes and compared under matched conditions.

The main evaluation uses a producer-side factorial benchmark campaign. This campaign varies broker count, replication factor, minimum in-sync replicas, message size, target throughput, producer count, acknowledgement mode, batch size, and compression type. This allows the artefact to examine whether TLS and mTLS introduce overhead, and how that overhead changes under different Kafka workload and durability settings.

A smaller targeted consumer-side validation study is also included. This does not reproduce the full producer factorial campaign. Instead, it evaluates whether secure transport also affects Kafka's read path under selected workloads, especially where mTLS processing interacts with larger messages and compressed payloads.

For each benchmark run, the artefact records application-level metrics, including throughput

and latency, alongside host-level telemetry from the benchmark client and Kafka brokers. This includes CPU utilisation, memory utilisation, disk activity, and network activity. Raw benchmark logs are parsed into structured JSON and CSV outputs, allowing reproducible comparison of plaintext, TLS, and mTLS results.

1.6 Dissertation Structure

The remainder of this dissertation is organised as follows. Chapter 2 reviews the literature on Apache Kafka, Kafka performance, Kafka security, Zero Trust Architecture, TLS/mTLS overhead, and benchmarking methodology. It identifies the research gap addressed by this dissertation.

Chapter 3 describes the design, implementation, and testing of the Kafka benchmarking artefact. It covers the infrastructure design, Kafka deployment model, security-mode configuration, factorial workload generation, orchestration scripts, telemetry collection, result-processing pipeline, and validation strategy.

Chapter 4 presents and discusses the benchmark results. It compares plaintext, TLS, and mTLS under matched workloads and analyses the observed throughput, latency, and host-telemetry behaviour.

Chapter 5 evaluates the dissertation objectives, summarises the main findings, discusses limitations, and identifies directions for future work.

Chapter 2

Literature Review

2.1 Introduction to Literature Review

This literature review examines the areas needed to understand the performance impact of Zero Trust-aligned transport security in Apache Kafka. It does not treat Kafka and Zero Trust as separate topics. Instead, it focuses on the interaction between Kafka's high-throughput architecture and security mechanisms such as TLS and mutual TLS (mTLS), which add cryptographic processing to Kafka's communication paths.

The chapter first reviews Apache Kafka as a distributed event-streaming platform, including its producer, broker, consumer, topic, partition, and replication model. It then analyses the mechanisms that contribute to Kafka's performance, such as append-only logs, batching, compression, page-cache utilisation, partition-level parallelism, pull-based consumption, and zero-copy data transfer. These mechanisms are important because they form the baseline against which TLS and mTLS overhead must be measured.

The review then examines Kafka's native security mechanisms, including encryption, authentication, and authorisation. This leads into a discussion of Zero Trust Architecture, where a key distinction is made: Zero Trust is a system-level security paradigm, while Kafka only provides mechanisms that can support Zero Trust-aligned deployments. In this dissertation, TLS and mTLS are therefore evaluated as measurable transport-security mechanisms aligned with Zero Trust principles, rather than as a complete implementation of Zero Trust Architecture.

The chapter then reviews existing work on TLS and mTLS performance overhead, followed by benchmarking methodologies for Kafka and distributed systems. This is necessary because existing TLS studies often focus on web servers or generic client-server systems, while Kafka has a replicated, partitioned, and batch-oriented data path. Finally, the chapter synthesises these areas into the research gap addressed by this dissertation: the lack of a reproducible and Kafka-specific empirical evaluation of plaintext, TLS, and mTLS

under matched high-throughput configurations.

2.2 Apache Kafka & Distributed Streaming Systems

Distributed streaming systems, such as Apache Kafka, are designed to store, process, and move continuous streams of data across distributed applications [3, 4, 6]. Kafka is widely used because it provides fault tolerance, high availability, reliability, and high-throughput event delivery [3, 4, 17]. These properties make it suitable for cloud and enterprise systems where many applications must exchange data reliably and at scale.

Apache Kafka is built around a cluster of servers called brokers and client applications called producers and consumers [3, 4]. Producers publish events to Kafka topics, while consumers subscribe to and read those events from the brokers [3, 4]. Brokers therefore act as the intermediary layer between data-producing and data-consuming applications. A simplified version of this architecture is shown in Figure 2.1.

Kafka stores events in topics, which are implemented as append-only, time-ordered logs [3, 6, 17]. Topics are divided into partitions, and these partitions can be distributed across multiple brokers. This allows Kafka to scale horizontally because different brokers and clients can process different partitions in parallel [3, 4, 6]. Replication further improves availability and durability by storing copies of partition data across multiple brokers [3, 6].

Kafka also uses a pull-based consumption model. Instead of brokers pushing events to consumers, consumers fetch records at their own rate [6]. This decouples producers from consumers and allows consumers to process large batches without being directly overwhelmed by the production rate. This model contributes to Kafka's scalability and fault tolerance, but it also means that performance depends heavily on client behaviour, batching, partitioning, and broker configuration.

These design choices create important trade-offs. Kafka performance is affected by acknowledgement settings, replication, batching, partitions, broker configuration, and client configuration [6, 7, 13, 14, 17]. For example, stronger durability settings can increase latency because producers may need to wait for more replicas to acknowledge a write. Similarly, poor partitioning or unsuitable broker configuration can limit throughput even when the underlying platform is capable of higher performance.

Therefore, Kafka performance should not be treated as a fixed property of the platform. It is shaped by the selected configuration, deployment environment, workload, and coordination costs [6,7,13,14,17]. This is important for this dissertation because security mechanisms such as TLS and mTLS add communication and cryptographic processing to Kafka's distributed data path. Their overhead must therefore be evaluated within Kafka's architecture, rather

than treated as a generic network cost [7, 13, 14, 18].

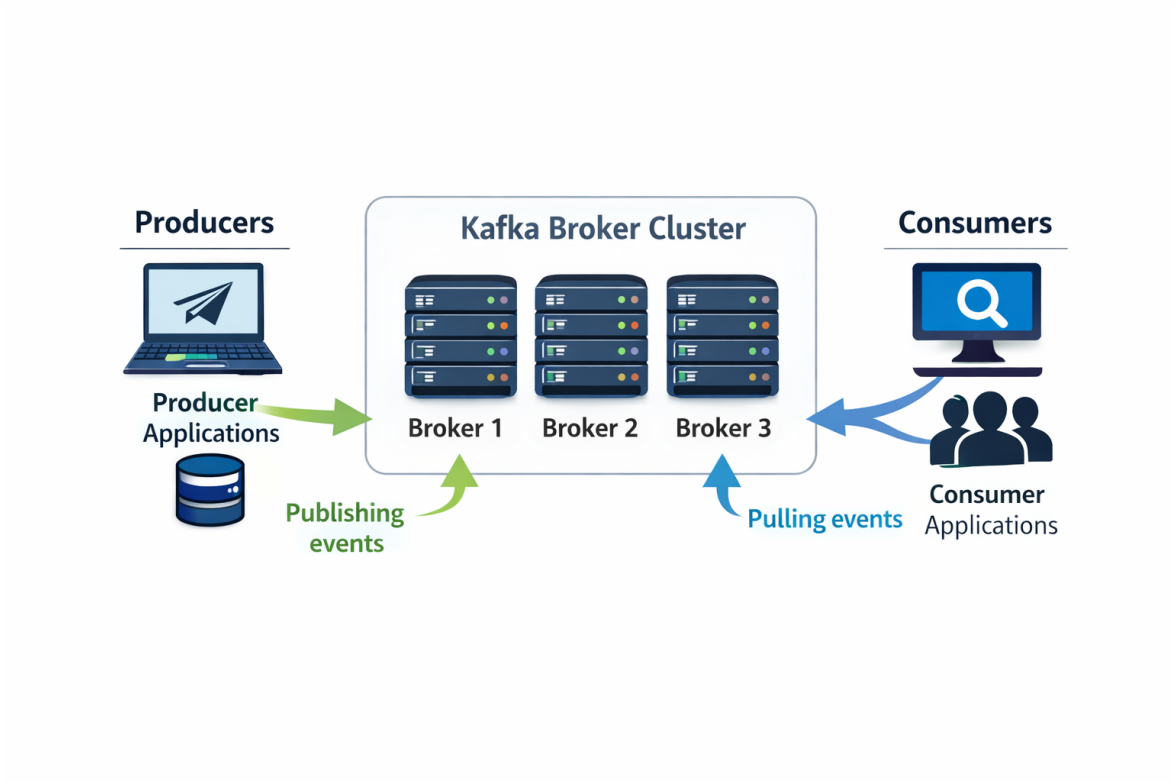

Figure 2.1: Simplified Apache Kafka architecture showing producer applications publishing events to a cluster of brokers, and consumer applications pulling events from the cluster. Figure generated using ChatGPT image generation from a prompt requesting a producer-broker-consumer Apache Kafka architecture diagram, April 2026.

2.3 Performance in Kafka Systems

Kafka performance comes from several architectural choices that reduce unnecessary data movement and allow data to be processed in parallel. Instead of relying on random-access data structures such as B-Trees, Kafka stores records in append-only logs, allowing efficient sequential writes [3, 6, 17]. Kafka also relies heavily on the operating system page cache and uses zero-copy I/O, specifically the `sendfile` system call, to transfer data from the page cache to the network socket with minimal copying [6]. These optimisations make Kafka fast, but they also make it sensitive to mechanisms that move processing back into user space, such as TLS and mTLS [6, 13, 18, 19]. This contrast is shown in Figure 2.2.

Kafka also gains performance through partitioning, batching, compression, and pull-based consumption [3, 4, 6, 17]. Topics are divided into partitions, which can be distributed across brokers and accessed in parallel by producers and consumers [3, 4, 6]. Producers can group records into larger batches, reducing network round-trip overhead and improving throughput [6, 7, 17]. Consumers then fetch records using a pull-based model, allowing them to read batches at their own processing rate [4–6, 14].

Compression further changes Kafka's performance behaviour. By compressing batches, producers can reduce network usage and broker storage pressure [6, 7, 14]. However, this also shifts some computational cost to producers and consumers, especially during compression and decompression. This matters for this dissertation because TLS and mTLS also introduce cryptographic processing. Therefore, the combined effect of compression and transport security may depend on CPU availability, message size, and client concurrency.

Kafka configuration choices also create explicit throughput-latency-durability trade-offs [6, 7, 13, 14, 17]. For example, producers can use different acknowledgement settings. Waiting for all in-sync replicas improves durability, but can increase latency compared with weaker acknowledgement settings [6, 7, 13]. Similarly, replication factor, minimum in-sync replicas, batch size, and producer concurrency can all change the balance between throughput, latency, and reliability. This means Kafka performance is not a fixed property of the platform, but an outcome of workload and configuration choices.

This relationship can also be understood using a simple queueing-theoretic model. In publish-subscribe systems, average latency (L) can be expressed in terms of processing time (τ) and system utilisation (U) as:

$$L = \frac{\tau}{1 - U} \tag{2.1}$$

[4, 5, 19]. As processing time increases, utilisation moves closer to saturation, and latency can grow non-linearly rather than linearly [4, 5, 17, 19]. This is important because TLS and mTLS can increase processing time by adding encryption, decryption, and certificate-based authentication to Kafka communication paths [6, 13, 18, 20].

Therefore, Kafka performance must be evaluated under controlled configurations rather than assumed from general platform claims. The impact of transport security may vary with batching, compression, producer count, replication, acknowledgement mode, message size, and broker resources. This motivates the need to examine Kafka's native security mechanisms before measuring their performance impact in the artefact.

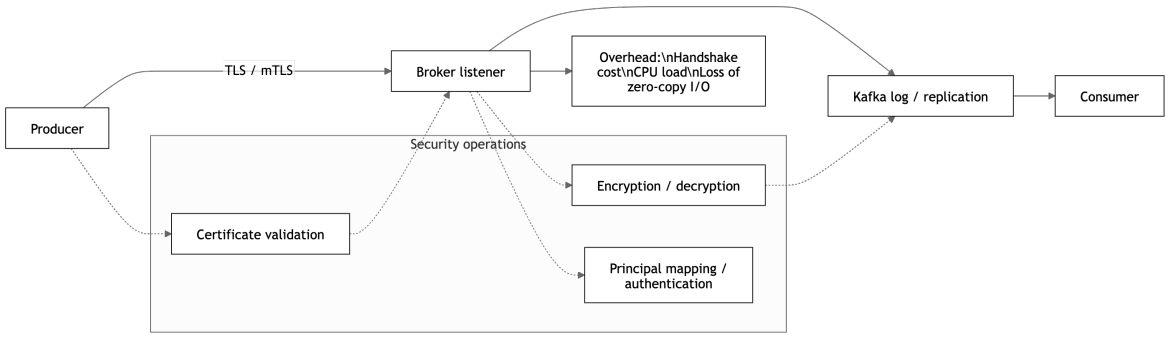

Figure 2.2: Comparison of Kafka's native zero-copy plaintext path and the TLS/mTLS path. In plaintext mode, Kafka can use `sendfile()` to transfer data efficiently from the page cache to the socket. Under TLS/mTLS, this optimisation is disabled, forcing data through user space for encryption before transmission.

2.4 Security Mechanisms in Kafka

Kafka provides native security mechanisms for protecting data streams, including encryption, authentication, and authorisation [3, 7, 13–15]. These mechanisms are important because Kafka is often deployed across distributed services where producers, brokers, and consumers exchange sensitive data. However, enabling security can also change Kafka's data path by adding cryptographic processing, identity verification, and access-control checks [6, 7, 13, 14].

Encryption in transit is provided through Transport Layer Security (TLS). TLS protects data moving between Kafka clients and brokers, and can also be configured for broker-to-broker communication [7, 13, 14]. Kafka exposes this through listener and SSL configuration options, including `ssl.enabled.protocols` and `ssl.cipher.suites`, which allow operators to control the supported TLS protocols and cipher suites [7, 13, 14]. This provides confidentiality and protection against eavesdropping or man-in-the-middle attacks, but it also introduces encryption and decryption work into Kafka's communication path.

This matters for performance because Kafka's plaintext path can use efficient operating-system mechanisms such as `sendfile`. Kafka's design documentation states that this zero-copy optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. Therefore, TLS does not only add cryptographic cost; it can also change how data moves between disk, memory, and the network. This makes Kafka-specific measurement necessary, because generic TLS overhead results from web servers or simple client-server systems do not fully represent Kafka's replicated and batch-oriented architecture [17–19].

Kafka can also support mutual TLS (mTLS). In this mode, clients present certificates

to brokers, and brokers verify those identities using Kafka SSL configuration such as `ssl.client.auth` [7, 13, 14]. This allows producers, consumers, and brokers to authenticate using cryptographically signed X.509 certificates. A simplified view of this security path is shown in Figure 2.3.

mTLS is especially relevant to Zero Trust-aligned deployments because it provides encrypted communication and certificate-based identity verification. However, it can also increase the amount of cryptographic work in the system. In Kafka, this work may occur not only on client-broker communication, but also on broker-to-broker traffic when internal replication is secured [6, 13, 20]. Reconnection events can also create additional TLS or mTLS handshake load, especially when many clients reconnect at the same time [13,18,19].

Therefore, Kafka's native security mechanisms create a direct performance-security trade-off. They improve confidentiality, authentication, and access control, but they may also increase CPU usage, latency, and data movement through user space. For this reason, TLS and mTLS are the main focus of this dissertation. They are measurable transport-security mechanisms that align with Zero Trust principles, while also directly affecting the Kafka data path that enables high throughput.

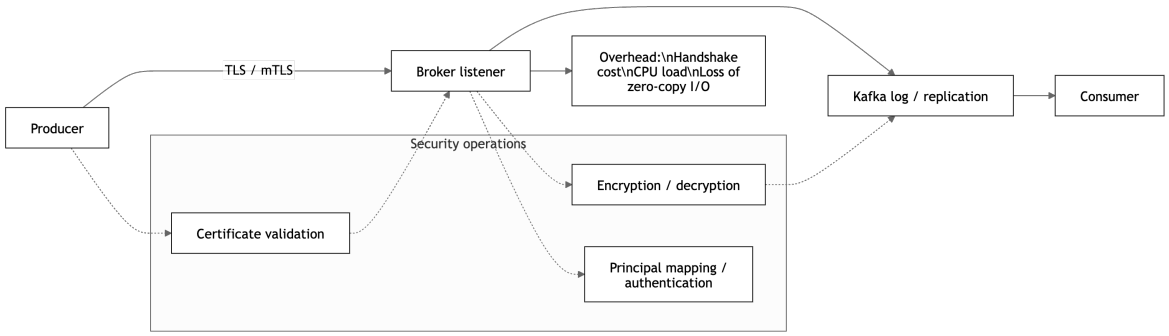

Figure 2.3: Kafka security data path illustrating how TLS/mTLS, certificate-based identity verification, principal mapping, and authorisation checks are applied across client-broker and broker-to-broker communication.

2.5 Zero Trust Architecture

Zero Trust Architecture (ZTA) is a security model developed in response to the limits of traditional perimeter-based security [8–10, 12]. Traditional models often assume that systems inside the network boundary are trustworthy. ZTA rejects this assumption and instead follows the principle of "never trust, always verify" [8–11]. In this model, access is

continuously evaluated using identity, context, policy, and device or service state, rather than being granted simply because a request originates from inside the network.

The core principles of ZTA include strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained policy enforcement [8–11]. These principles are commonly implemented through architectural components such as Policy Decision Points (PDPs) and Policy Enforcement Points (PEPs), which separate policy decisions from policy enforcement [8, 9, 11]. ZTA is therefore not a single tool or protocol, but a system-level security approach.

Cryptographic transport security is one part of this wider model. TLS and mTLS are often used to protect communication channels and verify service identities in distributed systems [8–10, 20]. TLS provides encrypted communication and server authentication, while mTLS extends this by requiring both sides of a connection to authenticate with certificates. This makes mTLS especially relevant to Zero Trust-aligned machine-to-machine communication, where services should not be trusted purely because they are inside the same network.

However, ZTA also introduces operational and performance challenges. Implementing it can require changes to identity management, certificate handling, monitoring, network segmentation, and access-control policy design [8–12]. In high-throughput systems, these controls can add processing and coordination overhead. This creates the key tension for this dissertation: the same mechanisms that improve trust and identity assurance may also affect the latency and throughput of systems such as Apache Kafka [6, 18, 19].

2.6 Zero Trust in Distributed Streaming Systems

The challenges of Zero Trust Architecture also apply to distributed streaming systems [1, 8–10, 12]. Modern event-streaming pipelines often span cloud services, remote devices, multi-cloud environments, and third-party networks. In these settings, components cannot be assumed to be trustworthy simply because they are inside the same network. Each producer, broker, stream processor, and consumer may need to be authenticated, authorised, encrypted, and monitored [8–10, 12]. Figure 2.4 illustrates this type of control model.

This creates a direct tension for streaming systems. These systems are designed for continuous data movement, low latency, and high throughput. However, Zero Trust-aligned controls such as mutual authentication, encryption, segmentation, and policy enforcement add work to the same communication paths that carry the data [12, 18–20]. Therefore, the main issue is not whether these controls are useful, but how their cost changes under realistic streaming workloads.

Access control is one example of this tension. Role-Based Access Control (RBAC) is widely used because it is simple to understand and operate, but it can lack the granularity required in dynamic distributed systems [8–10]. Attribute-Based Access Control (ABAC) provides finer-grained control by evaluating attributes such as identity, device state, time, location, and environmental context [8, 9, 12]. However, ABAC can be harder to configure correctly, and run-time policy evaluation can add latency if it is placed on the critical data path [8–10, 12].

Transport security creates a similar issue. Some studies suggest that modern TLS can have low steady-state overhead when connections are persistent and hardware is adequately provisioned [15, 19]. However, this does not fully resolve the problem for streaming systems. Event-streaming workloads may involve large message volumes, many clients, broker-to-broker communication, compressed batches, and reconnect events. In these cases, TLS and mTLS overhead may depend on workload shape, connection behaviour, certificate verification, and available CPU resources [12, 18–20].

This is particularly important for Apache Kafka because Kafka is designed around efficient batching, partitioning, replication, and zero-copy data transfer [6, 17]. Zero Trust-aligned transport controls, especially TLS and mTLS, can interact with these mechanisms by moving work into user space and adding cryptographic processing to client-broker and broker-to-broker communication. Therefore, the performance impact cannot be inferred from generic TLS studies alone. It must be measured in Kafka under controlled and reproducible configurations.

This establishes the need for the empirical approach used in this dissertation. The relevant question is not whether Zero Trust principles are valuable, but what performance cost is introduced when selected Zero Trust-aligned mechanisms are applied to a high-throughput streaming platform. This motivates the comparison of plaintext, TLS, and mTLS Kafka deployments under matched workloads.

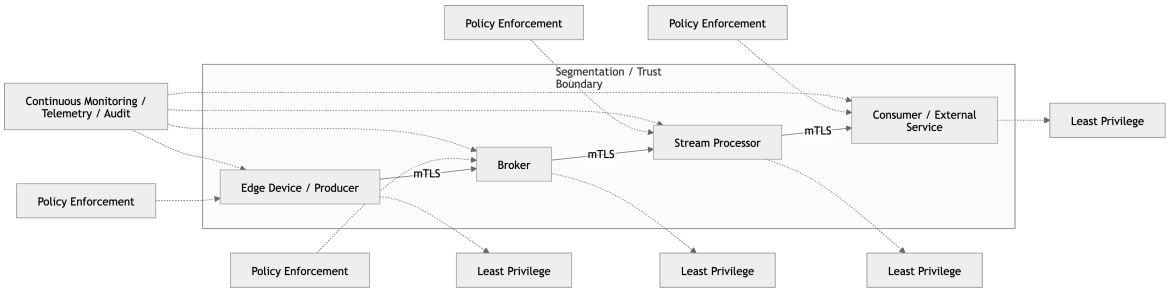

Figure 2.4: Conceptual Zero Trust control model for a distributed streaming pipeline, illustrating how mTLS, policy enforcement, least-privilege access, and continuous monitoring are applied across producer, broker, stream-processing, and consumer components within a segmented trust boundary.

2.7 TLS/mTLS Performance Overhead

Existing TLS performance studies show that encrypted transport can introduce measurable computational overhead [18,19]. Coarfa et al. separate this cost into connection setup, which uses asymmetric cryptography, and bulk data transfer, which uses symmetric encryption [18]. Their web-server experiments report a throughput degradation factor between 3.4 and 9 compared with plaintext baselines, depending on workload and session reuse [18]. They also identify RSA-based operations as a major source of overhead, consuming between 13% and 58% of web-server processing time under the evaluated workloads [18].

However, these findings cannot be transferred directly to Apache Kafka. Traditional web-server TLS benchmarks often model short-lived request-response workloads, where new connections are frequently created and where data is already handled in user space [18, 19]. Kafka is different. It is based on long-lived client-broker connections, batching, partitions, replication, and operating-system optimisations such as page-cache reuse and zero-copy I/O [6, 7, 14]. Kafka's design documentation states that `sendfile` is not used when SSL is enabled because SSL libraries operate in user space [6]. This means TLS affects Kafka not only through encryption cost, but also by changing the data-movement path.

Persistent-connection studies give a different view of TLS overhead. Iorio et al. show that persistent encrypted channels can reduce repeated handshake costs, allow TCP behaviour to stabilise, and make steady-state TLS latency small on sufficiently provisioned devices [19]. This is important because Kafka also uses long-lived connections. However, these studies still do not fully represent Kafka because they focus on generic client-server communication rather than replicated, broker-mediated streaming systems [6, 19].

In Kafka, TLS and mTLS may apply across several communication paths. These include

producer-to-broker, consumer-to-broker, and broker-to-broker communication [3, 6, 13]. When mTLS is enabled, certificate-based authentication can also be required in addition to encryption [13, 20]. Therefore, the cost is not limited to a single client-server channel. It may interact with replication, batching, compression, partitioning, and broker resources [6, 7, 13, 14].

The Kafka benchmarking literature also shows a reproducibility problem. Mohammad reports that although many Kafka studies include empirical benchmarks, they often omit important configuration details such as partition count, replication factor, batch size, linger time, compression settings, and workload characteristics [17]. This makes it difficult to compare results across studies, because observed performance differences may come from configuration changes rather than security overhead alone [7, 13, 17].

Therefore, the performance impact of TLS and mTLS in Kafka should be treated as configuration-dependent. It depends on how cryptographic work is distributed across clients and brokers, how it interacts with batching and compression, and how close the system is to CPU, network, or disk saturation [6, 7, 13, 18]. This motivates the need for a controlled benchmarking framework that compares plaintext, TLS, and mTLS under matched Kafka configurations.

2.8 Benchmarking Methodologies

Performance evaluation in distributed systems often starts with theoretical models. Network studies may use topological metrics such as bisection bandwidth or sparsest cut to estimate throughput limits [21]. Kafka studies may also use queueing models to relate latency to processing time and utilisation [5]. These models are useful because they explain why saturation can increase latency and reduce throughput. However, they are not sufficient on their own.

Jyothi et al. show that cut-based metrics do not always predict worst-case throughput, since a topology with a stronger cut metric can still perform worse under certain traffic patterns [21]. Similarly, Kafka producer evaluations show that queueing models can overestimate performance because they do not fully capture operating-system overhead, thread coordination, CPU contention, and network I/O contention [5]. Therefore, theoretical models are useful for framing the problem, but empirical benchmarking is required to measure the real behaviour of Kafka under load [6, 17].

Latency measurement also needs care. Some empirical studies rely on network-level round-trip time (RTT), using tools such as ping, traceroute, or transport-level measurements [19]. These metrics can show whether the network path is stable, but they do not capture the full application-level path. Iorio et al. argue that application-level

Flow Completion Time (FCT) is more representative because it includes protocol processing, endpoint behaviour, buffering, proxying, and other effects that network-level probes can miss [19].

This matters because distributed systems can appear stable at median latency while still showing large tail-latency increases. Iorio et al. show that performance can remain similar up to high percentiles but diverge at the extreme tail due to shared infrastructure, endpoint processing, buffering, and transport behaviour [19]. Therefore, Kafka benchmarking should not rely only on average or median latency. It should also capture maximum latency and, where available, high-percentile behaviour.

Kafka benchmarking also suffers from reproducibility issues. Researchers use standardised suites such as TPCx-Kafka, synthetic frameworks such as the Yahoo Streaming Benchmark, and custom workload generators [17]. Standardised suites provide recognisable baselines, while custom workloads can model more domain-specific pipelines such as finance, IoT, retail, or machine-learning systems [12, 15, 17]. However, Mohammad's review of 42 Kafka event-streaming studies shows that benchmark reporting is often incomplete and inconsistent [17].

This lack of transparency is a major problem for security-overhead evaluation. Kafka performance depends on parameters such as partition count, replication factor, message size, batch size, compression, producer concurrency, consumer lag, and acknowledgement settings [7, 13, 17]. If these are not reported or controlled, then performance differences may be caused by configuration variation rather than TLS or mTLS overhead. Sundbom also shows that aggregate metrics can hide internal client-side effects, such as producer queueing and outbuffer contention [5].

Therefore, evaluating Kafka transport-security overhead requires a transparent and reproducible benchmarking method. The benchmark must use explicit configuration files, matched workloads, structured output, and enough metric detail to connect application-level behaviour with resource-level causes. This motivates the artefact developed in this dissertation, which uses controlled factorial benchmark plans to compare plaintext, TLS, and mTLS under matched Kafka configurations.

2.9 Synthesis & Research Gap

The literature shows that Apache Kafka is a high-throughput distributed event-streaming platform whose performance depends on several connected mechanisms. These include append-only logs, page-cache utilisation, batching, partition-level parallelism, pull-based consumption, compression, replication, and zero-copy data transfer [3, 6, 7, 14]. These mechanisms allow Kafka to move large volumes of data efficiently, but they also make

performance sensitive to configuration choices and additional processing in the data path [5–7, 13, 17]. Therefore, Kafka performance should be treated as a configurable outcome, not as a fixed platform property.

The literature also shows that modern distributed systems increasingly require security models that do not rely on implicit trust inside the network [8–10]. Zero Trust Architecture addresses this by emphasising strong identity, least-privilege access, continuous verification, encrypted communication, monitoring, and fine-grained control [8–10]. However, Zero Trust is not a single feature that can be enabled in Kafka. Instead, Kafka provides mechanisms that can support Zero Trust-aligned deployments, including TLS encryption, mTLS-based authentication, SASL authentication, ACL authorisation, and broker-to-broker security configuration [7, 13, 14].

Among these mechanisms, TLS and mTLS are the most relevant to this dissertation because they directly affect Kafka's data path. They provide a measurable basis for comparing plaintext, server-authenticated, and mutually authenticated transport configurations [6, 13]. This creates a performance-security tension. Kafka's plaintext path can rely on efficient operating-system mechanisms such as `sendfile`, but Kafka's design documentation states that this optimisation is not used when SSL is enabled because SSL libraries operate in user space [6]. TLS and mTLS can therefore introduce both cryptographic processing and additional user-space data movement.

Existing TLS studies provide useful evidence, but they do not fully answer the Kafka-specific problem. Web-server studies show that TLS can introduce measurable overhead, especially during connection setup and asymmetric cryptographic operations [18, 20]. Persistent-connection studies show that this overhead can be smaller once handshake costs are amortised [19]. However, Kafka is not a stateless web server or a single client-server channel. It is a replicated, partitioned, batch-oriented distributed log with producer-broker, consumer-broker, and broker-to-broker communication paths [3, 6, 13]. Therefore, Kafka-specific measurement is required.

Kafka performance studies also do not fully resolve the problem. Existing work often examines throughput, batching, replication, producer parallelism, or benchmark-suite design without isolating TLS, mTLS, client authentication, and encrypted broker communication under matched configurations [5, 13, 17]. Where Kafka security is discussed, evaluations are often domain-specific, configuration-limited, or insufficiently reproducible [15, 17]. This limits the ability to compare results across workloads and deployment conditions.

The benchmarking literature further strengthens this gap. Kafka performance depends on variables such as broker count, replication factor, producer concurrency, batch size, acknowledgement mode, compression, message size, and partitioning [6, 7, 13]. If these

parameters are not controlled or reported, observed differences may come from configuration variation rather than security overhead. Coarse network metrics and theoretical models are also insufficient on their own because they do not fully capture operating-system overhead, thread coordination, network I/O contention, or application-level latency [5, 19, 21]. Therefore, evaluating Kafka security overhead requires transparent, repeatable, and configuration-explicit benchmarking.

The research gap addressed by this dissertation is therefore the lack of a reproducible, systematic, and Kafka-specific benchmarking framework for quantifying the performance overhead of Zero Trust-aligned transport-security mechanisms, particularly TLS and mTLS, under controlled high-throughput configurations [6, 13, 17]. This dissertation addresses that gap by designing and evaluating a configurable Kafka benchmarking artefact that provisions cloud-based Kafka clusters, executes matched plaintext, TLS, and mTLS workloads, records producer-side throughput and latency, captures host-level telemetry from clients and brokers, and includes targeted consumer-side validation.

The contribution is both methodological and empirical. Methodologically, the artefact provides a reproducible approach for evaluating Kafka transport-security overhead under matched conditions. Empirically, it provides evidence about how TLS and mTLS affect throughput, latency, and resource utilisation across Kafka workload and deployment parameters.

Chapter 3

Design, Implementation, and Testing

3.1 Chapter Overview

This chapter describes the design, implementation, and testing of the Kafka benchmarking artefact developed to answer the research question. The artefact provides a reproducible way to measure the performance overhead introduced by Zero Trust-aligned transport-security mechanisms, particularly TLS and mTLS, in Apache Kafka deployments.

The chapter first translates the research aim into concrete artefact requirements. It then describes the overall system architecture, including the local orchestration environment, Terraform-based AWS infrastructure, Kafka broker cluster, benchmark client, and result-processing workflow. This is followed by the Kafka deployment model, including the use of KRaft mode, broker readiness checks, and the separation between public SSH access and private Kafka traffic.

The chapter also explains how the three security modes are implemented. Plaintext is used as the baseline, TLS provides encrypted server-authenticated transport, and mTLS extends this with certificate-based client authentication. These modes are not treated as a complete Zero Trust Architecture. Instead, they are evaluated as measurable transport-security mechanisms that align with Zero Trust principles such as encrypted communication and strong identity verification.

The benchmark design is then presented. This includes the producer-side factorial campaign, the targeted consumer-side validation study, the JSON-driven configuration model, deterministic run identifiers, and the use of matched workloads across security modes. The chapter also explains the telemetry pipeline used to collect application-level metrics and host-level resource measurements from the benchmark client and Kafka brokers.

Finally, the chapter describes the result-processing, export, fault-tolerance, and testing strategy. This includes structured JSON and CSV outputs, checkpointed execution, per-run topic cleanup, readiness validation, and smoke testing across plaintext, TLS, and mTLS modes. Together, these sections show how the artefact was designed to support reproducible, controlled, and comparable evaluation of Kafka transport-security overhead.

3.2 Artefact Requirements

The artefact requirements were derived from the research question and the methodological gap identified in Chapter 2. Since the dissertation evaluates the performance overhead of TLS and mTLS in Kafka, the artefact must support controlled deployment, matched workload execution, structured metric capture, and repeatable comparison across security modes. The requirements are divided into functional requirements, which describe what the artefact must do, and non-functional requirements, which describe the qualities needed for a valid benchmarking framework.

The core functional requirements are as follows:

1. Provision cloud-based Kafka infrastructure using repeatable infrastructure-as-code.
2. Deploy Kafka clusters in plaintext, TLS, and mTLS configurations.
3. Execute matched producer workloads across security modes.
4. Execute targeted consumer-side validation workloads.
5. Capture application-level performance metrics, including throughput and latency.
6. Capture host-level telemetry from Kafka brokers and benchmark clients.
7. Parse raw benchmark logs into structured result files.
8. Export results in formats suitable for analysis and dissertation reporting.

The core non-functional requirements are as follows:

1. Reproducibility, so benchmark runs can be regenerated from explicit configuration files.
2. Comparability, so plaintext, TLS, and mTLS runs use matched workload and deployment parameters.
3. Resumability, so long-running campaigns can continue after interruption without rerunning completed work.
4. Fault tolerance, so transient failures and interrupted runs do not corrupt local state.

5. Configuration transparency, so all workload and deployment parameters are recorded with each result.
6. Cost control, so cloud resources and benchmark scope remain manageable.
7. Extensibility, so additional workloads, metrics, or security modes can be added without redesigning the framework.

Requirement	Description
Reproducibility	Benchmark runs must be generated from explicit configuration files and produce structured outputs.
Comparability	Plaintext, TLS, and mTLS runs must use matched workload and deployment parameters.
Resumability	Long-running campaigns must recover from interruption without rerunning completed rows.
Fault tolerance	Deployment and benchmark execution must avoid corrupting state when interrupted or retried.
Telemetry	Each run must capture application metrics and host-level resource utilisation.
Configuration transparency	Each result must record the workload, deployment, and security parameters used for the run.
Cost control	The benchmark design must remain feasible on cloud infrastructure by using controlled campaign sizes and reusable deployments.

Table 3.1: Core artefact requirements

These requirements shaped the artefact design. Infrastructure provisioning was automated to support reproducibility, benchmark execution was driven by explicit configuration files to support comparability, and checkpointing was added to support long-running campaigns. Host-level telemetry was also included because throughput and latency alone are insufficient to explain whether observed overhead is caused by CPU pressure, network activity, disk activity, or client-side bottlenecks.

3.3 System Architecture

The artefact uses a cloud-based architecture so that Kafka can be evaluated on virtual machines rather than on local containers. This was important because the dissertation evaluates not only Kafka application metrics, but also host-level CPU, memory, disk, and network behaviour. The system consists of a local workstation, Terraform infrastructure code, orchestration scripts, an AWS Virtual Private Cloud (VPC), Kafka broker instances, and a dedicated benchmark client.

Figure 3.1 shows the high-level architecture. The local workstation acts as the control node. It stores the Terraform configuration, orchestration scripts, benchmark configuration files, generated factorial plans, checkpoints, and copied result files. Terraform provisions the AWS infrastructure, while the orchestration scripts prepare the benchmark client, deploy Kafka to the brokers, configure the selected security mode, execute benchmark runs, collect telemetry,

and copy results back to the local results directory.

The AWS environment contains Kafka broker EC2 instances and one benchmark client EC2 instance. The broker cluster supports both three-broker and five-broker experimental phases, allowing the benchmark design to evaluate different broker-count configurations. Kafka traffic uses the brokers' private VPC IP addresses, while SSH orchestration uses public IP addresses. This separation is important because benchmark traffic should represent internal cluster communication rather than external internet routing.

The benchmark client is responsible for running the Kafka producer and consumer benchmark tools. It connects to the brokers using the appropriate Kafka listener for the selected security mode: plaintext, TLS, or mTLS. During each run, application-level benchmark output is collected alongside host-level telemetry from the benchmark client and brokers. The results are then copied back to the local workstation, where they are parsed, aggregated, and exported into structured outputs for analysis.

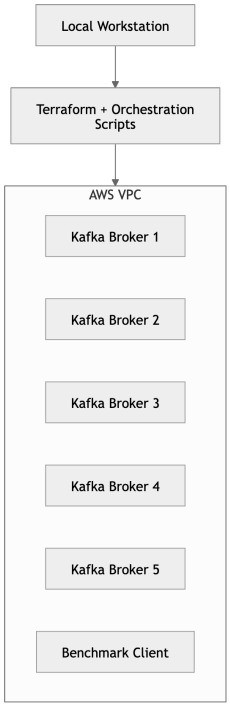

Figure 3.1: High-level system architecture of the Kafka benchmarking artefact.

3.4 Infrastructure Provisioning

The infrastructure layer was implemented using Terraform and AWS EC2. Terraform was selected because it allows the cloud environment to be described as infrastructure-as-code, making the deployment repeatable and easier to inspect. This was important for the artefact

because benchmark results should be linked to a clearly defined environment rather than an informal manual setup.

The Terraform configuration provisions an AWS Virtual Private Cloud (VPC), subnet, security groups, Kafka broker instances, and a dedicated benchmark client instance. The VPC provides an isolated network for the experiment, while security groups control access to SSH, Kafka broker listeners, and internal cluster communication. Public IP addresses are used for orchestration over SSH, while Kafka traffic uses private IP addresses inside the VPC. This avoids routing benchmark traffic through the public internet and keeps the measured Kafka communication closer to an internal cloud deployment.

The broker infrastructure supports both three-broker and five-broker phases. This distinction was needed because broker count is one of the experiment variables, and a three-broker run should not be treated as equivalent to a five-broker run with two unused brokers. Separate broker-count phases allow the artefact to evaluate Kafka under different cluster sizes while keeping the active deployment consistent with the benchmark configuration.

A separate benchmark client instance is provisioned alongside the brokers. This instance runs the Kafka producer and consumer benchmark tools, starts benchmark workloads, and collects client-side output. Separating the benchmark client from the brokers prevents the load generator from competing directly with broker processes on the same host. It also makes the client a measurable part of the experiment, since producer and consumer CPU usage can contribute to the observed overhead of TLS and mTLS.

Each EC2 instance uses a 40 GB gp3 root volume. This was added because early benchmark runs showed that retained Kafka topic data could exhaust smaller broker disks during large-message workloads. The larger root volume, combined with per-run topic cleanup, reduces the risk that disk exhaustion affects benchmark validity or interrupts long-running campaigns.

Virtual machines were selected instead of containers because the experiment required control over machine-level resource behaviour and host telemetry. Containers would have made local development and deployment faster, but they would also add another abstraction layer around CPU scheduling, network I/O, disk I/O, and memory accounting. Since the dissertation evaluates not only throughput and latency but also CPU, memory, disk, and network activity, EC2 virtual machines provide a more appropriate environment for observing host-level behaviour.

This infrastructure design therefore supports the requirements of reproducibility, comparability, and telemetry. Terraform provides repeatable provisioning, EC2 provides

measurable host environments, and the separated broker/client topology allows Kafka performance to be evaluated under controlled cloud conditions.

3.5 Kafka Cluster Deployment

Kafka was deployed in KRaft mode rather than using a separate ZooKeeper ensemble. In KRaft mode, Kafka brokers participate in the cluster metadata quorum directly, reducing the number of external services required for the artefact. This made the deployment simpler and more suitable for automated provisioning, while still reflecting the current direction of Kafka cluster management.

Each broker instance is configured as part of the Kafka cluster and assigned a broker identifier. The brokers form a controller quorum, which is responsible for cluster metadata such as topic definitions, partition assignments, broker membership, and leadership changes. The orchestration scripts generate the required cluster metadata, including the controller quorum configuration and private broker addresses, before starting Kafka on each instance.

Kafka broker traffic is configured to use private VPC IP addresses. This is important because producer, consumer, broker-to-broker, and controller communication should occur over the internal cloud network rather than through public IP addresses. Public IP addresses are used only for SSH orchestration from the local workstation. This separation reduces external network variation and keeps benchmark traffic within the intended experimental environment.

Topics are created for benchmark runs using explicit workload parameters. Each run defines the topic name, partition count, replication factor, and minimum in-sync replicas. Partition count controls the degree of parallelism available to producers, consumers, and brokers. Replication factor controls how many broker copies of each partition are maintained. The `min.insync.replicas` setting defines the minimum number of replicas that must acknowledge writes when stronger acknowledgement settings are used. Together, these parameters directly affect Kafka's throughput, latency, and durability behaviour.

Benchmark topics are created as part of each run rather than reused indefinitely. This allows the run metadata to be linked to a specific topic configuration and reduces the risk that previous benchmark state affects later measurements. Topic cleanup is also performed after runs where appropriate, which prevents retained test data from filling broker disks during long-running campaigns.

Broker readiness was validated using Kafka API checks rather than relying only on `systemd` process status. A broker process can be running while the broker is not yet ready to serve Kafka requests, participate correctly in the cluster, or respond to metadata operations.

The readiness checks therefore verify Kafka-level availability before benchmark execution begins. This reduces the risk of starting producer or consumer workloads against brokers that are alive at the process level but not operational at the Kafka API level.

This deployment design supports repeatable benchmark execution by making cluster state, topic configuration, and broker readiness explicit. It also ensures that workload parameters such as partitions, replication factor, and `min.insync.replicas` are treated as controlled experimental variables rather than incidental deployment details.

3.6 Security Mode Design

The artefact supports three Kafka transport-security modes: plaintext, TLS, and mutual TLS (mTLS). These modes were selected because they allow the benchmark to compare an unsecured baseline, an encrypted server-authenticated configuration, and an encrypted mutually authenticated configuration under matched workloads. This makes it possible to isolate the overhead introduced by transport encryption and client certificate authentication.

Plaintext mode is used as the baseline. In this mode, Kafka clients and brokers communicate without transport encryption or certificate-based authentication. This does not represent a secure production deployment, but it provides the reference point needed to measure the additional cost introduced by TLS and mTLS.

TLS mode enables encrypted communication between Kafka clients and brokers. In this configuration, the broker presents a certificate and the client verifies the broker identity. This provides confidentiality and server authentication, while still allowing the benchmark to measure the overhead of encrypted transport without requiring client certificates.

mTLS mode extends TLS by requiring client certificate authentication. In this configuration, clients also present certificates to the broker, allowing both sides of the connection to be authenticated. This mode is the closest of the three benchmark configurations to a Zero Trust-aligned transport model because it combines encrypted communication with certificate-based identity verification.

These modes do not represent a complete Zero Trust Architecture. A complete Zero Trust deployment would also require broader policy enforcement, continuous monitoring, least-privilege authorisation, identity lifecycle management, and operational controls beyond Kafka transport configuration. Instead, TLS and mTLS are evaluated in this dissertation as measurable Zero Trust-aligned transport-security mechanisms.

The TLS and mTLS configurations permit TLS 1.3 and TLS 1.2, with protocol negotiation handled by the Kafka/JDK SSL stack. This means the artefact does not currently isolate

TLS protocol-version overhead. The security-mode comparison therefore measures the overhead of the configured Kafka transport-security modes as deployed, rather than a TLS 1.3-only or TLS 1.2-only experiment.

Mode	Description
Plaintext	Baseline with no transport encryption.
TLS	Server-authenticated encrypted transport.
mTLS	Encrypted transport with client certificate authentication.

Table 3.2: Kafka transport-security modes evaluated by the artefact

3.7 Benchmark Workload Design

The benchmark workload design contains two parts: a producer-side factorial campaign and a targeted consumer-side validation campaign. The producer campaign is the main evaluation because the research question focuses on the overhead introduced into Kafka's high-throughput write path under plaintext, TLS, and mTLS configurations. The consumer campaign is included as a smaller validation study to check whether secure transport also affects Kafka's read path.

The producer benchmark uses Kafka's `kafka-producer-perf-test.sh` tool. This tool was selected because it is part of the Kafka distribution and provides direct measurements of producer throughput and latency under configurable workload parameters. The artefact extends this basic producer benchmark by adding structured run metadata, deterministic run identifiers, concurrent producer support, host telemetry, result parsing, aggregation, and checkpointed execution.

Producer concurrency is controlled through the `producer_count` parameter. When this value is greater than one, the benchmark client starts multiple producer processes for the same run. This allows the artefact to evaluate whether TLS and mTLS overhead changes when several producers generate load concurrently. This is important because cryptographic cost may appear differently under a single producer compared with a multi-producer workload.

The producer campaign varies the following parameters:

- `message_size_bytes`: size of each produced record.
- `target_messages_per_second`: target producer throughput.
- `batch_size`: producer batch size.
- `acks`: producer acknowledgement setting.

- `producer_count`: number of concurrent producer processes.
- `compression_type`: producer compression mode.
- `replication_factor`: number of broker replicas for benchmark topics.
- `min_insync_replicas`: minimum replicas required for acknowledged writes.
- `broker_count`: number of brokers used in the cluster phase.
- `security_mode`: plaintext, TLS, or mTLS.

These parameters were selected because they affect the throughput-latency-durability trade-off discussed in Chapter 2. For example, message size and target throughput affect the amount of data sent through the transport layer, while batching and compression affect how efficiently records are grouped before transmission. Acknowledgement mode, replication factor, and `min_insync_replicas` affect durability and broker coordination. Broker count and producer count affect the degree of parallelism available in the system. Security mode is the primary comparison variable used to estimate the overhead of TLS and mTLS.

The consumer-side benchmark was intentionally designed as a targeted validation slice rather than a full factorial campaign. This kept the evaluation tractable while still testing whether secure transport also affects Kafka's read path. A full consumer factorial campaign would significantly increase execution time and cloud cost, while duplicating many producer-side variables that are not necessary for a focused read-path validation.

The consumer validation campaign compares plaintext, TLS, and mTLS using selected message sizes, compression settings, consumer counts, and repeated trials. Its purpose is not to replace the producer factorial campaign, but to provide supporting evidence about whether the overhead observed on the write path also appears when records are consumed. This is particularly relevant because compression and secure transport can shift work between producers, brokers, and consumers.

Together, the producer and consumer workloads provide a balanced evaluation design. The producer campaign gives broad factorial coverage of Kafka write-path behaviour, while the consumer validation slice adds read-path evidence without making the overall experiment infeasible.

3.8 Factorial Experiment Design

The producer campaign was designed as a controlled factorial experiment. The purpose of this design was to evaluate how Kafka transport-security overhead changes across different workload and deployment conditions, rather than measuring only one fixed configuration. Each run is generated from explicit configuration files so that the workload, security mode,

and deployment parameters are recorded and reproducible.

The artefact separates experiment definition from execution. The factorial design is first described in JSON configuration files, where the parameter ranges, trial count, baseline values, and validity rules are defined. These JSON files are then expanded into a generated JSONL plan. Each line in the JSONL plan represents one benchmark row with a complete set of workload, deployment, and security parameters. This makes the final campaign auditable before execution, because the planned rows can be inspected without starting any cloud workload.

Each planned row is assigned a deterministic run identifier. The run identifier encodes the main workload and deployment parameters, including security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, and trial number. Deterministic identifiers support reproducibility and resumability because the same configuration always maps to the same run identity.

Validity constraints are applied during plan generation. These constraints remove Kafka configurations that would be invalid or misleading. For example, replication factor and `min_insync_replicas` must be compatible with the selected broker count. This prevents the executor from attempting runs that Kafka could not validly create, and ensures that comparisons are made only across meaningful configurations.

The final producer campaign design contains 5,184 planned rows after validity constraints are applied. This consists of 1,728 plaintext rows, 1,728 TLS rows, and 1,728 mTLS rows. The purpose of this balanced structure is to support matched security-mode comparison. For each valid workload and deployment configuration, the same configuration can be evaluated under plaintext, TLS, and mTLS, allowing overhead to be estimated relative to the plaintext baseline.

This design also supports repeated trials. Repetition is important because individual benchmark runs can be affected by transient cloud behaviour, JVM warm-up effects, background operating-system activity, and network variation. Using repeated trials provides a more reliable basis for comparing security modes than relying on a single run for each configuration.

The factorial design is intentionally controlled rather than exhaustive across every possible Kafka parameter. The selected parameters were chosen because they are directly relevant to throughput, latency, durability, and cryptographic overhead. This keeps the campaign large enough to evaluate different Kafka operating conditions, while still remaining feasible within the time and cost constraints of a cloud-based dissertation artefact.

3.9 Execution and Orchestration

The benchmark execution layer is implemented as a set of orchestration scripts. These scripts connect the infrastructure layer, Kafka deployment, benchmark client, telemetry collection, result parsing, aggregation, and export workflow. This design keeps the experiment reproducible because the same commands and configuration files can be used to regenerate plans, resume campaigns, and rebuild result summaries.

The first stage is plan generation. The `generate_factorial_plan.sh` script reads the factorial JSON configuration and expands it into a JSONL execution plan. Each line in this plan represents one complete benchmark run. This separates experiment design from execution: the planned campaign can be inspected, counted, filtered, and validated before any benchmark workload is started.

The main execution script is `run_factorial_plan.sh`. This script reads the generated JSONL plan, applies optional filters such as security mode or broker count, and executes benchmark rows that have not already been completed. For each row, it prepares the run metadata, creates the benchmark topic, starts telemetry collection, runs the producer or consumer benchmark command on the benchmark client, copies the output back to the local machine, parses the result, and records completion state.

The executor records campaign state using structured JSONL files. The `started.jsonl` file records runs that have begun execution, while `completed.jsonl` records runs that have finished successfully. If a run fails, the failure information is written to `failures.jsonl`. A separate checkpoint file is also maintained so that long-running campaigns can resume without rerunning rows that have already completed. This is important because the final campaign contains thousands of planned rows and may be interrupted by network issues, SSH failures, local process termination, or deliberate pauses.

After each run, parser scripts convert raw Kafka benchmark logs and telemetry files into structured result data. The per-run output includes the raw producer or consumer benchmark log, run metadata, host telemetry, and a parsed `result.json`. The parser extracts application-level metrics such as throughput and latency, links them to the workload configuration, and records telemetry summaries where available.

Aggregation scripts then combine individual `result.json` files into campaign-level outputs. The main aggregated outputs are `summary.json` and `summary.csv`. The JSON summary preserves structured metadata and nested metric fields, while the CSV summary provides a flat tabular format for spreadsheet analysis, plotting, and statistical comparison.

Export scripts are used after aggregation to produce dissertation-ready outputs. These include comparison tables, CSV exports, LaTeX tables, and plots. This separation

between execution, parsing, aggregation, and export makes the pipeline easier to validate because raw logs remain available even if the summarisation or export logic needs to be rerun.

Overall, the orchestration design supports resumability, auditability, and reproducibility. Each run has explicit metadata, each campaign has recorded state, and each result can be traced back from exported summaries to the raw Kafka benchmark output.

3.10 Telemetry and Metric Collection

The artefact collects both application-level metrics and host-level telemetry. This was necessary because throughput and latency show the observable benchmark outcome, but they do not explain why a run behaves differently under plaintext, TLS, or mTLS. Host telemetry provides additional evidence about whether overhead is associated with CPU utilisation, memory usage, network traffic, or disk activity.

The application-level metrics are collected from the Kafka benchmark tools. For producer runs, the main metrics are throughput in records per second, throughput in MB per second, average latency, and maximum latency. These metrics are parsed from the raw Kafka benchmark output and stored in each run's structured result file. They provide the main basis for comparing plaintext, TLS, and mTLS performance.

Host-level telemetry is collected from the benchmark client and Kafka broker instances during benchmark execution. The telemetry includes CPU utilisation, memory utilisation, network receive/transmit activity, and disk read/write sector counters. Collecting telemetry from both client and broker hosts is important because TLS and mTLS overhead may appear on either side of the connection. For example, encryption, decryption, compression, producer batching, and certificate verification can affect the benchmark client, the brokers, or both.

Each run also records workload and deployment metadata. This includes the run identifier, security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, trial number, and topic configuration. This metadata is needed so that results can be grouped, filtered, and compared under matched conditions.

At campaign level, the aggregation process records the telemetry host count and failure count. The telemetry host count shows how many machines contributed host-level measurements for the run or campaign. The failure count is used to separate successful benchmark rows from failed or incomplete rows, preventing failed executions from being silently mixed into performance summaries.

Some result files also include interval-derived p95 and p99 fields. These fields are treated as diagnostic indicators rather than true per-record latency percentiles. This distinction is important because they are derived from interval-level benchmark output rather than from a complete per-record latency distribution. They can help identify tail-latency behaviour, but the dissertation does not treat them as exact per-record percentile measurements.

The combined metric set therefore supports both comparison and explanation. Throughput and latency measure the application-level effect of each security mode, while host telemetry helps relate those effects to resource utilisation. This is central to the dissertation because the research question is not only whether TLS and mTLS introduce overhead, but also how that overhead appears across Kafka clients, brokers, and workload configurations.

3.11 Result Processing and Export

The artefact separates raw benchmark execution from result processing. Each benchmark run produces a run directory containing the raw Kafka benchmark output, workload metadata, telemetry files, and parsed result data. This structure preserves the original evidence for each run while also providing structured outputs for aggregation and analysis.

The raw benchmark logs are kept so that parsed results can be checked against the original Kafka tool output. For producer runs, this includes the raw producer performance log. For consumer runs, this includes the corresponding consumer benchmark output. Topic creation and cleanup logs are also retained where available. Keeping these files is important because it makes the result-processing pipeline auditable rather than relying only on derived summaries.

Each run also contains a `metadata.json` file. This file records the workload, deployment, and security parameters used for the run, including security mode, broker count, replication factor, minimum in-sync replicas, message size, target throughput, batch size, acknowledgement mode, producer count, compression type, trial number, and topic details. This ensures that every result can be traced back to the exact configuration that produced it.

The parser converts the raw benchmark output and metadata into a structured `result.json` file. This file contains the parsed application-level metrics, workload metadata, telemetry summaries, and execution status for a single run. It acts as the canonical per-run result used by the aggregation scripts.

At campaign level, aggregation scripts combine all valid `result.json` files into `summary.json` and `summary.csv`. The `summary.json` file preserves structured fields and nested metadata, making it suitable for programmatic analysis. The `summary.csv` file provides a flat table that can be inspected manually, loaded into spreadsheet tools, or used for statistical analysis.

The export layer then produces dissertation-ready outputs from the aggregated summaries. These include CSV tables, LaTeX tables, and SVG plots. The CSV exports provide clean tabular data, the LaTeX tables can be inserted directly into the dissertation, and the SVG plots provide scalable figures for visual comparison. Security comparison exports are also generated to compare plaintext, TLS, and mTLS results under matched workload configurations.

This processing design supports reproducibility and transparency. Raw logs remain available for verification, per-run `result.json` files provide structured evidence, and campaign-level summaries support comparison across large benchmark sets. The export layer then converts these results into forms suitable for Chapter 4 analysis and dissertation presentation.

3.12 Fault Tolerance and Idempotency

Fault tolerance and idempotency were important design requirements because the benchmark campaigns are long-running and executed on remote cloud infrastructure. A full campaign can be interrupted by SSH failures, local process termination, cloud instability, benchmark errors, or deliberate pauses to control cost. The artefact therefore needed to recover from interruption without corrupting state or repeating completed work unnecessarily.

Deployment checkpointing is used during infrastructure and Kafka preparation. The deployment scripts record which stages have already completed, such as inventory generation, benchmark client preparation, broker bootstrapping, Kafka configuration, and readiness validation. If a deployment step is interrupted, the process can resume from the recorded state rather than repeating the entire deployment from the beginning.

Benchmark checkpointing is used during factorial execution. The executor records run state in `started.jsonl`, `completed.jsonl`, `failures.jsonl`, and a checkpoint file. Before starting a run, the executor checks whether the deterministic run identifier has already been completed. If it has, the run is skipped. This prevents completed rows from being rerun when a campaign is resumed.

Each run uses a unique deterministic run identifier. This identifier is derived from the workload, deployment, security mode, and trial parameters. Unique run IDs make result directories predictable and prevent results from different configurations being mixed together. They also make it possible to trace each row in the generated plan to its raw logs, metadata, parsed result, and summary entry.

Local state writes are handled carefully to avoid partial or corrupted files. Where

possible, scripts write to temporary files or temporary directories first, then move the completed output into its final location. Result copying also uses temporary local directories before a run is marked as complete. This reduces the risk that an interrupted file transfer leaves an incomplete result directory that is later treated as valid.

Topic cleanup is included to prevent retained benchmark data from filling broker disks. Benchmark topics are created for runs using explicit parameters and deleted after the run where appropriate. This per-run topic deletion was added after early large-message workloads showed that retained data could exhaust broker storage. Combined with larger broker root volumes, topic cleanup improves the reliability of long-running campaigns.

Retry wrappers are used around remote operations that may fail transiently, such as SSH commands, file transfers, and service checks. These retries reduce the chance that short-lived network or remote-host issues cause an entire campaign to fail. However, failures are still recorded when a run cannot complete, so that failed rows can be inspected rather than silently ignored.

Kafka readiness checks are performed at the API level rather than relying only on `systemd` service status. A Kafka process can be running while the broker is not yet ready to handle metadata requests or serve benchmark traffic. API readiness checks reduce the risk of starting benchmark workloads before the cluster is actually operational.

Together, these mechanisms make the artefact resilient to interruption and suitable for repeated execution. Deployment checkpoints, benchmark checkpoints, atomic writes, unique run identifiers, topic cleanup, temporary result copying, retry wrappers, and API readiness checks all support a precise and methodical benchmarking process.

3.13 Testing and Validation

Testing was carried out at multiple levels because the artefact combines infrastructure provisioning, Kafka deployment, security configuration, benchmark execution, telemetry collection, and result processing. The purpose of testing was not only to check whether individual scripts ran, but also to validate that the full benchmarking pipeline could produce structured and comparable results.

Infrastructure testing first checked whether the Terraform configuration could provision the required AWS resources. This included the VPC, subnet, security groups, Kafka broker instances, benchmark client instance, and storage volumes. This validation was necessary because benchmark correctness depends on a repeatable cloud environment.

Kafka deployment testing then focused on broker availability. Broker readiness was

checked using Kafka API-level validation rather than relying only on `systemd` process status. This confirmed that brokers were not only running as Linux services, but were also able to respond to Kafka metadata and client operations.

Security-mode testing was performed using smoke tests for plaintext, TLS, and mTLS. The plaintext smoke test validated the baseline producer path with no transport encryption. The TLS smoke test validated encrypted server-authenticated transport. The mTLS smoke test validated encrypted transport with client-certificate authentication. These tests confirmed that the benchmark client could connect to the brokers using the expected listener and security configuration before larger campaign execution.

Consumer-side testing was also performed to validate the read-path benchmark. The consumer smoke test confirmed that benchmark topics could be populated and then consumed successfully under the selected security mode. The mTLS consumer validation slice was completed successfully, showing that the consumer-side path could execute with telemetry and structured result capture.

Resumability was tested through checkpointed execution. The factorial executor records started, completed, and failed runs, then skips deterministic run identifiers that are already marked complete. This was validated by resuming campaigns after interruption and confirming that completed rows were not rerun.

Heavy-row testing was used to validate difficult workload configurations before committing to larger campaigns. These tests included larger message sizes and concurrent producer settings, which are more likely to expose broker, client, disk, or network limits. This helped confirm that the artefact could handle more demanding benchmark rows rather than only small smoke-test workloads.

Parser and aggregation testing confirmed that raw Kafka benchmark logs could be converted into structured outputs. Per-run logs were parsed into `result.json`, and campaign-level aggregation produced `summary.json` and `summary.csv`. This testing was important because Chapter 4 depends on structured metrics rather than manual interpretation of raw benchmark logs.

Table 3.3 summarises the main testing types used during artefact development.

Test type	Purpose
Terraform validation	Confirm infrastructure can be provisioned.
Broker readiness checks	Confirm Kafka API availability before benchmark execution.
Plaintext smoke test	Validate the baseline producer path.
TLS smoke test	Validate encrypted server-authenticated transport.
mTLS smoke test	Validate encrypted transport with client-certificate authentication.
Consumer smoke test	Validate the read-path benchmark.
Resume test	Confirm checkpointed execution skips completed rows.
Heavy-row test	Confirm large-message and concurrent-producer workloads can execute.
Parser test	Confirm raw logs are converted into structured JSON and CSV outputs.

Table 3.3: Testing and validation activities used for the benchmarking artefact

At the current stage of development, the artefact has passed plaintext historical validation, TLS smoke testing, mTLS smoke testing, and the mTLS consumer validation slice. The mTLS producer campaign phase is in progress, with completed rows reporting no recorded failures so far. Final campaign counts and performance results are reported in Chapter 4, rather than in this design chapter, because those values may continue to change during execution.

3.14 Chapter Summary

This chapter described the design, implementation, and testing of the Kafka benchmarking artefact used to evaluate transport-security overhead. The artefact was designed around the requirements of reproducibility, comparability, resumability, telemetry, and configuration transparency. These requirements shaped the use of Terraform-based AWS provisioning, EC2 broker and benchmark client instances, Kafka KRaft deployment, explicit workload configuration, and structured result capture.

The chapter also described how plaintext, TLS, and mTLS modes are implemented and compared. Plaintext provides the baseline, TLS adds encrypted server-authenticated transport, and mTLS extends this with client certificate authentication. These modes are evaluated as Zero Trust-aligned transport-security mechanisms rather than as a complete Zero Trust Architecture.

The benchmark framework combines producer-side factorial workloads, targeted consumer-side validation, host-level telemetry, checkpointed orchestration, fault-tolerant result handling, and exportable summaries. Testing and validation were used to confirm

that infrastructure provisioning, Kafka readiness, security-mode connectivity, benchmark execution, resumability, and result parsing worked as intended.

The following chapter uses the outputs of this artefact to evaluate the performance overhead of TLS and mTLS under matched Kafka workloads.

Chapter 4

Results and Discussion

This is where you present your findings (e.g., your throughput and latency data). To reach the 70+ band, your results must be presented in a logical order alongside a "perceptive and critical discussion" of what the data actually means

Chapter 5

Evaluation, Conclusions, and Further Work

Your final chapter must evaluate how well you satisfied your original objectives, provide an insightful summary of your conclusions, and discuss appropriate directions for future research

Bibliography

[1] A. Kumar, "Distributed systems in modern enterprise architecture: Challenges and solutions," *IJSAT*, vol. 16, no. 1, p. 2447, 2025.

[2] "Powered by," https://kafka.apache.org/powered-by/, accessed: 2026-03-24.

[3] "Introduction," https://kafka.apache.org/42/getting-started/introduction/, accessed: 2026-03-24.

[4] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85 333–85 350, 2023.

[5] A. Sundbom, "Parallel kafka producer applications: Their performance and its limitations," Master's thesis, KTH Royal Institute of Technology, 2023.

[6] "Design," https://kafka.apache.org/42/design/design/, accessed: 2026-03-24.

[7] "Producer configs," https://kafka.apache.org/42/configuration/producer-configs/, accessed: 2026-03-26.

[8] H. Kang, G. Liu, Q. Wang, L. Meng, and J. Liu, "Theory and application of zero trust security: A brief survey," *Entropy*, vol. 25, no. 12, p. 1595, 2023.

[9] N. F. Syed, S. W. Shah, A. Shaghaghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (zta): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57 143–57 179, 2022.

[10] V. A. Vora, "Demystifying zero trust security: The no-trust network paradigm," *Journal of Computer Science and Technology Studies*, vol. 7, no. 3, pp. 141–148, 2025.

[11] M. Tsai, S. Lee, and S. W. Shieh, "Strategy for implementing of zero trust architecture," *IEEE Transactions on Reliability*, vol. 73, no. 1, pp. 93–100, 2024.

[12] H. Bhoite, "Zero-trust architecture in streaming dataflows," https://www.authorea.com/doi/full/10.36227/techrxiv.175303721.12297807, accessed: 2026-03-26.

[13] "Broker configs," https://kafka.apache.org/42/configuration/broker-configs/, accessed: 2026-03-26.

[14] "Consumer configs," https://kafka.apache.org/42/configuration/consumer-configs/, accessed: 2026-03-26.

[15] J. Meka, "Financial services cloud transformation: Securing sensitive data in kafka event streams," *Journal of Computer Science and Technology Studies*, vol. 7, no. 4, pp. 1023–1028, 2025.

[16] A. A. Süzen and O. Ceylan, "The advantages and implementation challenges within the scope of the basic principles of transition to zero trust architecture," *International Journal of 3D Printing Technologies and Digital Industry*, vol. 8, no. 3, pp. 416–427, 2024.

[17] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," in *2025 5th International Conference on Intelligent Cybernetics Technology & Applications (ICICyTA)*, 2025, pp. 612–617.

[18] C. Coarfa, P. Druschel, and D. S. Wallach, "Performance analysis of tls web servers," *ACM Transactions on Computer Systems*, vol. 24, no. 1, pp. 39–69, 2006.

[19] M. Iorio, F. Risso, and C. Casetti, "When latency matters: Measurements and lessons learned," *SIGCOMM Computer Communication Review*, vol. 51, no. 4, pp. 2–13, 2021.

[20] C. H. Jack, S. K. Teck, L. T. Ming, and D. Y. Hong, "An overview analysis of authentication mechanism in microservices-based software architecture: A discussion paper," in *2023 IEEE 8th International Conference on Software Engineering and Computer Systems (ICSECS)*, 2023, pp. 1–6.

[21] S. A. Jyothi, A. Singla, P. B. Godfrey, and A. Kolla, "Measuring and understanding throughput of network topologies," in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 761–772.

Appendices

Appendix A

Ethics and GenAI Declaration

A.1 Ethics Statement

This dissertation uses a computer-based experimental methodology. The artefact provisions Apache Kafka clusters on AWS EC2, executes synthetic benchmark workloads, and records application-level performance metrics and host-level telemetry. The project does not involve human participants, user studies, interviews, surveys, behavioural observation, or the collection of personal data.

The benchmark data used in this dissertation is generated synthetically by Kafka producer and consumer tools. The recorded data consists of technical machine metrics, including throughput, latency, CPU utilisation, memory utilisation, disk activity, network activity, workload metadata, and benchmark execution status. No personally identifiable information is collected, processed, or analysed.

The project therefore does not require ethical approval for human-participant research. The main ethical and professional considerations relate to responsible cloud-resource usage, secure handling of credentials, and accurate reporting of experimental limitations. AWS credentials, SSH keys, certificates, and generated security material are excluded from version control where appropriate.

A.2 Generative AI Declaration

A.2.1 Acknowledge

Generative AI tools were used as support tools during this project. Their use included assistance with drafting and refining dissertation wording, reviewing structure against university guidance, improving clarity of technical explanations, and supporting software development planning. Generative AI was also used to create the initial Apache Kafka architecture diagram used in Chapter 2.

The project design, artefact implementation, benchmark execution, result interpretation, and final academic judgement remain the author's own work. AI-generated material was reviewed, edited, and adapted before inclusion.

A.2.2 Describe

Generative AI was used mainly for support and review rather than as a source of evidence. Its use included:

- suggesting clearer wording for dissertation sections;
- helping restructure paragraphs and chapter sections;
- reviewing whether sections aligned with the university marking criteria;
- helping explain technical trade-offs such as virtual machines versus containers;
- assisting with implementation planning and debugging strategy for the artefact;
- generating an initial Apache Kafka architecture diagram, which was then reviewed for relevance to the dissertation.

Generative AI was not used as an independent factual source. Technical claims were checked against cited literature, official Apache Kafka documentation, the artefact source code, and benchmark results. The author remains responsible for the accuracy, originality, and final form of all submitted work.

A.2.3 Evidence

Representative examples of GenAI use are summarised below.

Use case	Example prompt summary	How the output was used
Dissertation structure review	Asked for Chapter 1 and Chapter 2 to be reviewed against university marking criteria.	Used to identify missing structure, such as explicit aims, objectives, and clearer research gap framing.
Chapter wording support	Asked for sections to be rewritten in a clearer academic style while preserving the author's tone.	Outputs were edited and adapted before inclusion.
Artefact explanation	Asked for help explaining the Kafka benchmarking artefact, including infrastructure, security modes, telemetry, and result processing.	Used as drafting support for Chapter 3, checked against the implemented repository.
Technical planning	Asked for advice on benchmark design, fault tolerance, idempotency, and telemetry collection.	Used to support development planning; final implementation was tested directly.
Figure generation	Prompt requested a simplified Apache Kafka architecture diagram showing producers, brokers, topics, and consumers.	Output was used as Figure 2.1 and disclosed in the figure caption.

Table A.1: Representative examples of GenAI use during the project

Full prompt and response records were retained separately as supporting evidence and can be provided if required. These records are not reproduced in full in the dissertation to avoid excessive appendix length and to prevent inclusion of irrelevant implementation detail.

A.2.4 Declaration

The author confirms that no content created by generative AI technologies has been submitted without review, adaptation, and acknowledgement. The author remains responsible for the dissertation argument, artefact implementation, experimental execution, result interpretation, and final submitted content.

Appendix B

Artefact Configuration

Appendix C

Result Schema

Appendix D

Experiment Plans

Appendix E

Additional Result Tables

Appendix F

Repository and Reproducibility Notes